

试卷结构

- 1.选择题10题，每题2分 20%
- 2.简答题8题，每题5分 40%
- 3.综合分析题4题，每题10分 40%

Unit 1 概述

• 软件架构产生的背景

1、软件危机 2、统一软件开发视角，及早发现问题

• 软件危机、软件架构的作用

软件危机的根源：1、软件易变，需求不定 2、软件规模扩大 3、架构影响权重增加 4、提升开发效率

• 软件架构的思想和特征

软件架构的思想：**关注系统总体结构的组织**

软件架构的实现方式：1、利用抽象提升认知层次 2、建立“组件+连接件+配置”的结构组织方式

软件架构的特征：**1、注重可重用性 2、利益相关者较多 3、关注点分离 4、质量驱动 5、强调概念完整性 6、循环风格**

• 软件架构的发展阶段，各阶段特征

模块化计算与软件组件化技术的区别：

模块化指软件分成若干独立模块，单独开发，最后组合。

组件化技术指业务组件相互独立，组件与组件之间不存在直接的耦合关系。

组件化开发并不等同于模块化开发。模块化开发只是在逻辑上做了切分，物理上（开发出的系统代码）通常并没有真正意义上的隔离

• 软件架构研究和应用现状

- 软件架构描述语言：各种ADL
- 软件架构表示方法：形式化的架构描述方法、4+1”架构模型、UML、IEEE的软件架构描述规范

- 软件架构分析的目的是在系统被实际构造之前，预测其质量属性
- 软件架构的设计指生成一个满足软件需求的架构的过程
- 架构测试着重于仿真系统模型，解决架构层的主要问题

Unit 2 软件架构概念

软件架构的定义：

组成派和决策派的区别：

组成派关注于软件本身，将软件架构看做组件和交互的集合。

Merry shord

决策派关注于软件架构中的实体（人），将软件架构视为一系列重要设计决策的集合。

软件架构=组件+连接件+约束

参考定义框架：

- 软件架构=组件+连接件+配置+端口+角色
- 组件：可重用的软件模块单元
- 连接件：组件之间的交互
- 配置：组件和连接件的拓扑逻辑和约束
- 端口：组件和外部环境的交汇点
- 角色：连接件交互的参与者

Unit 3 软件架构模型

□ 软件架构模型是什么

软件架构建模是对架构设计决策的具象化和文档化。

意义在于它能够将软件架构某些关键或关注的方面剥离出来，使用统一的图形、文档和数据进行描述，便于理解交流。（剥离关键点，规范描述，便于交流）

简答题

□ 软件架构建模的五类方法（过程、优缺点）：

◦ 基于非规范的图形表示的建模方法

方法：将软件架构按照图形的方式进行表达。

两类：正式图形表示（树形结构），非正式图形表示（盒线图）

优点：便于理解，建模简单

缺点：不够规范，产生歧义

◦ 4+1 视图

画图

◦ 基于UML的建模方法

用例图、类图、对象图、状态图、协作图、顺序图、活动图、组件图、部署图

用UML对架构进行建模的三种途径：

- 将UML看作是一种软件架构描述语言直接对架构建模。
 - 简单，但有些架构概念无法对应。
 - 方法：建立基于UML的应用领域模型，然后建立非形式化的架构图。
 - 具体：定义类接口、定义连接件类、建立表示架构的类图、用协作图描述
- 扩展机制约束UML的元模型。
 - 增添新的结构而不改变现有的语法或语义，但工具支持较少。
 - 例如：基于UML的系统性能模型——早期给出性能评估，对设计方案定量描述，便于比较
- 对UML的元模型进行扩充。
 - 能满足架构建模需求，但与UML不兼容

优点：统一的表示方法、支持多视图、有效利用工具

缺点：架构的支持性不强，语义不够精准，信息冗余，非形式化的建模层次不够

◦ 基于形式化的建模方法

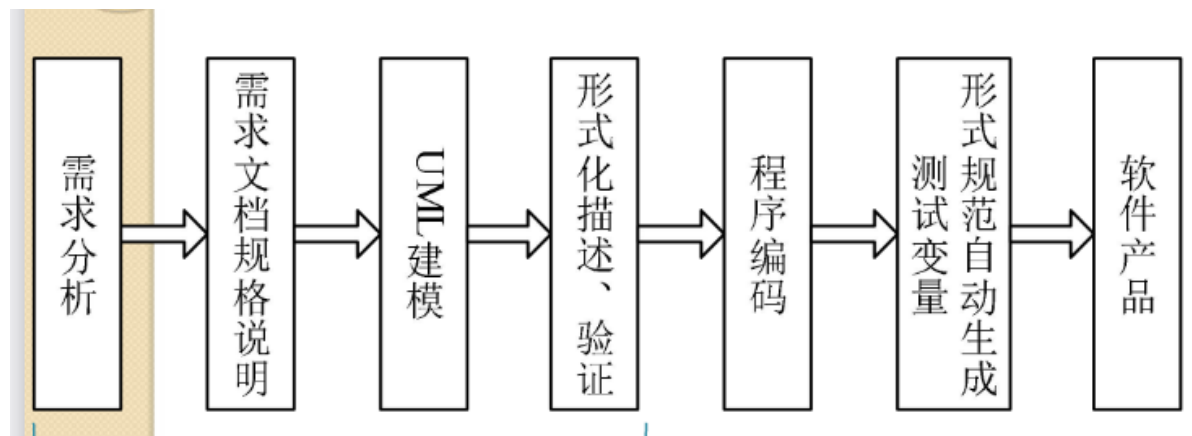
基本思想:利用一些已知特性的**数学抽象**来为目标软件系统的状态特征和行为特征构造模型。

◦ 基于UML形式化的方法

UML不是一种形式化的语言，不能精确的描述系统的运行语义。

类图、用例图、状态图以及顺序图较适合形式化描述。其他图形式化会更加复杂

建模过程：



◦ **其他建模方法（文本语言、MDA）** 选择、问答、简答

文本语言建模是通过特殊的句法描绘架构

优点：工具较多、开发支持完善

缺点：不易描述图形、显示方式单一

MDA：模型驱动架构

模型不仅被作为设计文档和规格说明来使用，更成为一种能够自动转换为最终可运行系统的重要软件。

核心思想是抽象出与实现技术无关、完整描述业务功能的平台独立模型

在MDA中，将模型区分为PIM（平台无关模型）和PSM（平台相关模型）

□ **"4+1"模型**

用例视图+逻辑视图+过程视图+开发视图+物理视图 分别对应哪些UML图

用例视图（场景）：从外部世界的角度描述正在建模系统的功能

逻辑视图：描述系统的组成方式和交互方式

过程视图：描述系统的进程

开发视图：系统的模块、组件管理

物理视图：软件如何映射到现实世界

视图：包含一组架构元素（软件或硬件）

Unit 4 软件架构风格与模式

简答

□ **什么是软件架构风格**

软件架构风格是**某一特定应用领域**中系统组织方式的**惯用模式**，作为“**可复用的组织模式和习语**”，为设计人员的交流提供了公共的术语空间，促进了**设计复用与代码复用**软件架构风格。

基本属性包括：设计元素的词汇表、配置规则、语义解释

简答

□ **使用架构风格的好处**

1. 促进设计的重用性和代码的重用性
 2. 系统的组织结构易被理解
 3. 更好支持系统内部的互操作性
 4. 便于分析
-

20种风格特点、优点、缺点、适用范围

数据流风格

特点：1、数据决定组件 2、系统结构由数据流向决定 3、组件之间基本只有数据交互

组件：数据处理单元

端口：输入端口 和 输出端口

连接件：数据流

角色：Reader和 Writer

批处理

组件：独立的应用程序

连接件：某种类型的介质，**完整**的数据

特点：近乎线性、每个处理是一个独立的程序、每一步在前一步结束后才开始、数据必须以整体的方式传播

管道过滤器

组件：过滤器

连接件：管道

场景：数据源源不断地产生，系统需要对这些数据进行若干处理

特点：各个组件相互独立、支持功能模块的复用、具有较强的可维护性和可扩展性、支持一些特定的分析、具有并发性

不足：大量处理过程、数据流进行解析或反解析复杂、交互弱

调用返回风格

主子程序

组件：主程序和子程序

连接件：调用-返回机制

场景：它适用于可以通过过程定义层次结构适当定义计算的应用

优点：结构清晰、开发过程逐步细化

不足：数据存储格式要求一致、规模不能太大、难以复用

面向对象

组件：对象

连接件：过程调用

场景：它适用于中心问题是识别和保护相关信息体（尤其是表示信息）的应用

优点：对象易于修改、完全性高、数据存取改为交互的程序集合

不足：对象太多、交互太多、行为责任分散、

层次结构

思想：系统被组织成若干个层次，每个层次由一系列组件组成，层次之间存在接口，通过接口形成 call/return 的关系——下层组件向上层组件提供服务，上层组件被看作是下层组件的客户端。

大多分为四层：表现层、业务层、持久层、数据库层

组件：程序的集合

连接件：调用过程

场景：它适用于涉及可分层排列的不同服务类别的应用程序。

特点：问题分解成渐进、不支持跨层交互、修改一层通常只会影响上层、上层必须了解下层

优点：复杂问题分解成一个增量步骤序列、层次影响小、支持层的重用

缺点：适用性窄、建模困难、性能受限

CS架构

两层CS：

- 优点：C与S分开，灵活性较高：数据、软件、硬件、整体成本较低
- 不足：S的成本高、需要专门程序、传输形式单一、难以扩展到网络、数据安全性低

三层CS：

- 相比两层进步：系统的逻辑结构清晰、可维护性强、环境选择性高、开发并行度高、安全性好
- 不足：通信效率问题、独立性要求

BS架构

优点：客户端操作简单、通信协议统一、开发成本低

缺点：个性化程度低、S的负载重、交互性较弱、可扩展性差、安全性不足、速度较慢

以数据为中心的风格

仓库风格

组件：中心数据结构组件（表示当前数据的状态）和相对独立的组件集合

连接件：数据仓库与独立组件之间的交互

场景：应用于核心问题是建立、扩充和维护一个复杂的中央信息体的情况

优点：便于模块间的数据共享、方便模块的添加、更新和删除、方便模块的添加、更新和删除

不足：需要一定的同步/加锁机制保证数据结构的完整性和一致性

黑板风格

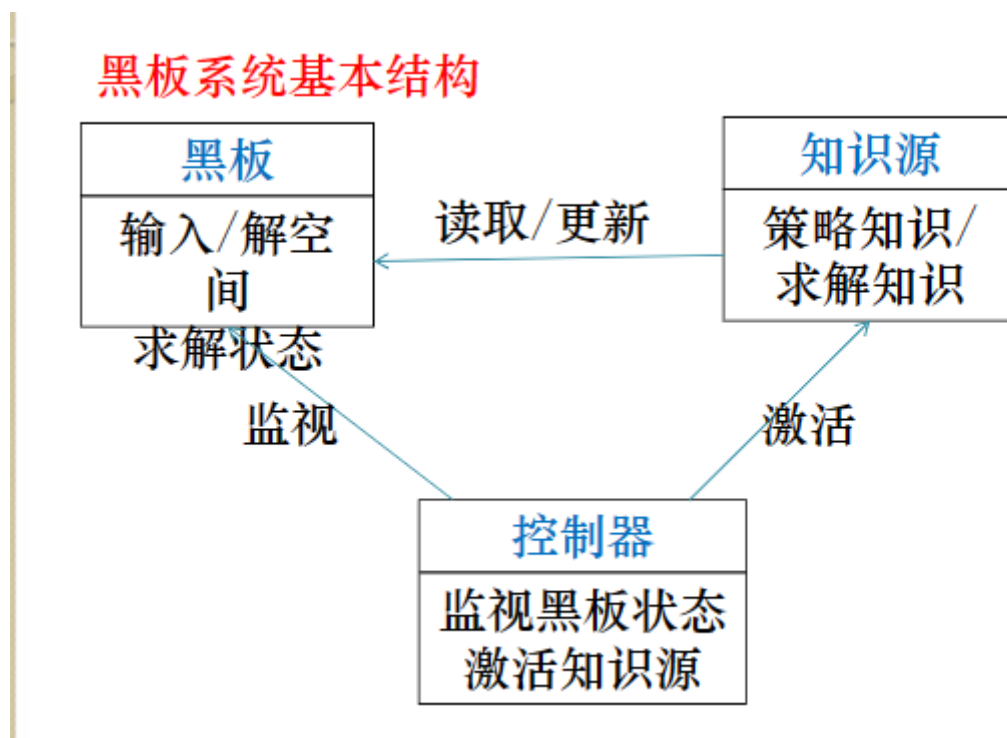
场景：一个大问题被分解为若干个子问题，每个子问题的解决需要不同的问题表达方式和求解模型，分别设计求解程序

组成：知识源、黑板、控制器

优点：多客户共享大量数据、易于扩展、知识源可重用、容错性

缺点：同步、数据结构要一致

方法：黑板存储解的状态，知识源独立，由控制器监视黑板状态启动知识源对黑板进行更新



虚拟机风格

解释器风格

基本思想：**执行**其他程序的程序，针对不同的硬件平台实现了一个虚拟机，将高抽象层次的程序翻译为低抽象层次所能理解的指令

优点：提升可移植性、便于模拟测试

不足：性能下降

解释器与编译器：

- 解释器会执行，编译器只会翻译
- 解释器更慢
- 解释器在解释时获取内存，编译器在编译时确定好内存分配

基于规则的系统风格

把频繁变化的业务逻辑抽取出来，形成独立的规则库。

业务逻辑=固定业务逻辑+可变业务逻辑(规则)+规则引擎

过程：1、定义规则 2、程序运行时根据状态选择规则 3、规则通过解释器翻译

优点：降低了修改业务逻辑的成本、规则共享

与解释器共同：在两个不同的抽象层次之间建立起一种虚拟的环境

与解释器不同：在规则和代码之间建立虚拟机；在代码和平台之间建立虚拟机

独立组件风格

事件驱动风格

基本思想：组件广播事件，其他组件通过监听事件产生隐式调用

方法：◦ 1) 确定响应事件的元素；
◦ 2) 为指定元素确定需要响应的事件类型；
◦ 3) 为该元素的相应事件编写事件处理程序；
◦ 4) 将事件处理程序绑定到指定元素的指定事件。

特点：事件的触发者并不知道哪些构件会被这些事件影响，相互保持独立

两种分派策略：广播式、选择广播式

优点：组件相对独立、组件复用性强、便于维护

不足：组件的掌控性弱、数据交换格式、正确性验证问题

其他架构风格

C2风格

基于组件和消息层次体系架构风格

规定：通过连接件绑定在一起的按照一组规则运作的**并行组件网络**。该规则规定了所有组件之间的交互必须通过**异步消息机制**来实现

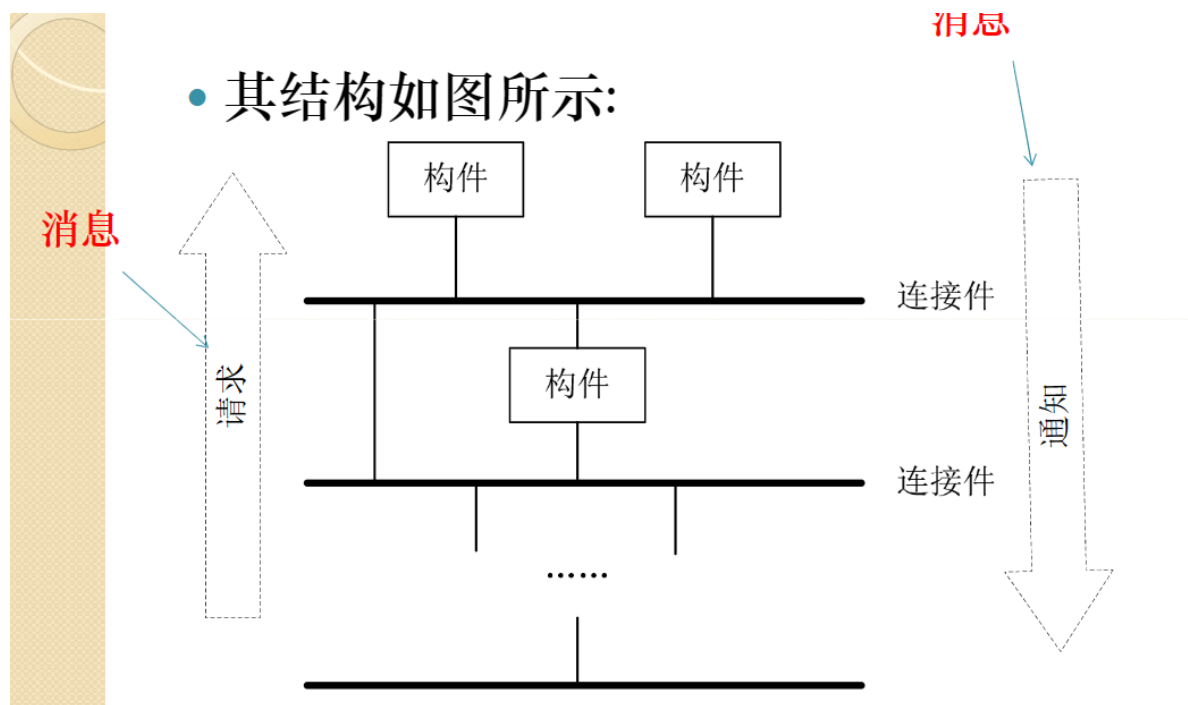
连接件：连接件负责把构件绑定在一起,其上可以连接任何数量的构件和接任何数量的构件和连接件

特点：通信和处理分开完成、组件与连接件必须顶部底部相接

优点：语言自由、组件易换、扩展性强、支持多用户多系统

不足：不适合数据库访问频繁

场景：适用于GUI软件开发，构建灵活和可扩展的应用系统



平台插件风格

插件的本质：不修改程序主体（或者程序运行平台）的情况下对软件功能进行扩展与加强

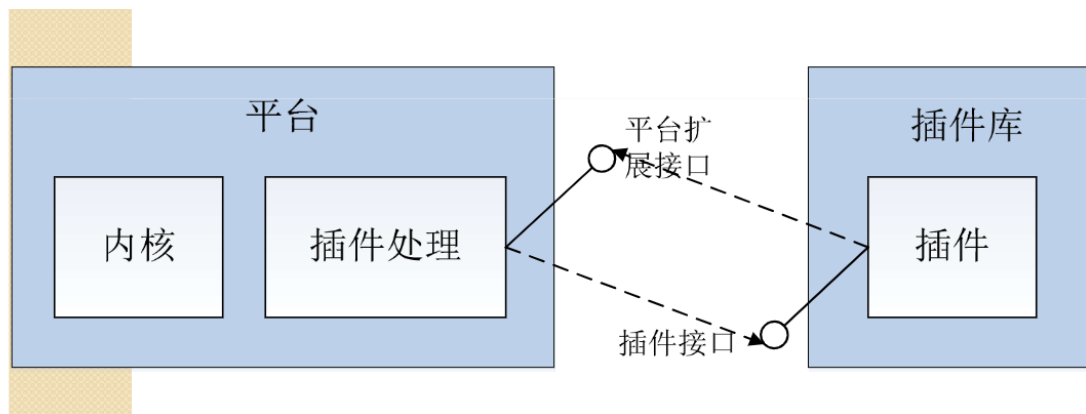
程序的主体或主框架，可定义为平台。又分为内核功能与插件处理功能

功能扩展或补充模块，可定义为插件。插件要不影响程序运行、对系统扩展、不与其他插件冲突。

两个接口：平台扩展接口。插件接口

优点：降低模块依赖性、各模块独立开发、系统可拆装

缺点：插件在不同平台可重用性差



面向Agent风格

特点：编程时由对象转向Agent智能体

基本思想：事物属性受环境影响。将事物以及影响因素结合为Agent作为系统的基本单位。

Agent：根据环境主动决策的软件实体

Agent组件：对系统处理的抽象，具有灵活智能的软件实体

Agent连接件：对复合型组件的连接，提供通信等服务。

Agent组件的不同：其所具有的自主性、智能性、交互性等特性是传统架构对象所不具备的。不同Agent之间的联系根据运行状态决定，并非固定连接件

优点：便于解决复杂问题，分布开放异构的软件环境

不足：大多数结构中Agent缺乏社会型结构描述与环境交互

面向方面风格

增加了方面组件来封装系统的横切关注点。

目的：分离“技术问题实现”和“业务问题实现”

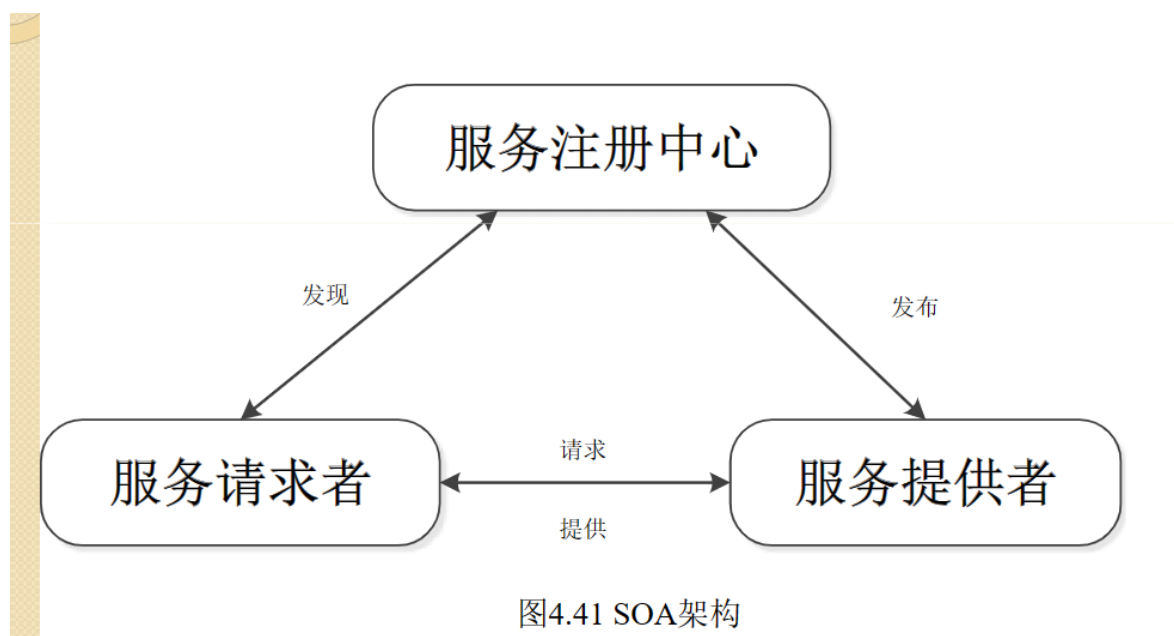
优点：关注点分离、与面向对象互补、对横切关注点模块化

面向服务风格

SOA 是一个组件模型，将服务通过接口联系起来。

优点：灵活、松耦合、易维护、粗粒度

不足：服务划分困难、接口标准不统一



微服务架构

将单个应用程序开发为一组小型服务，每个服务独立运行，通过网络通信。

解耦过程：功能剥离、数据解耦、数据同步

微服务架构是面向服务架构的一个变形

特点：围绕业务能力组织系统、自动化部署、程序和数据的去中心化控制、智能化

优点：各个模块独立部署、松耦合

缺点：分布后的复杂性、接口统一、代码重复、异步问题、维护麻烦

正交架构

正交软件架构由组织层(Layer)和线索(Thread)的组件(Component)构成。

基本思想：应用系统垂直分割为若干个线索(子系统)，线索又分为几个层次（系统有一个公共驱动层(一般为最高层)和公共数据结构(一般为最低层)）

层：由一组具有相同抽象级别(Level of Abstraction)的组件构成。

线索：子系统的特例，由完成不同层次功能的组件组成，完成整个系统的部分功能。

如果线索是相互独立的，即不同线索中的组件之间没有相互调用，那么这个结构就是完全正交的。

优点：结构清晰、易修改、可移植性好

不足：正交化的成本、适用性

异构风格

异构架构是几种风格的组合。

组合方式：使用层次结构。一个系统组件被组织成某种架构风格，但它的内部结构可能是成某种架构风格，但它的**内部结构**可能是另一种完全不同的风格

优点：代码重用、不同标准的组合

缺点：风格的兼容性

基于层次消息总线 (JB/HMB风格)

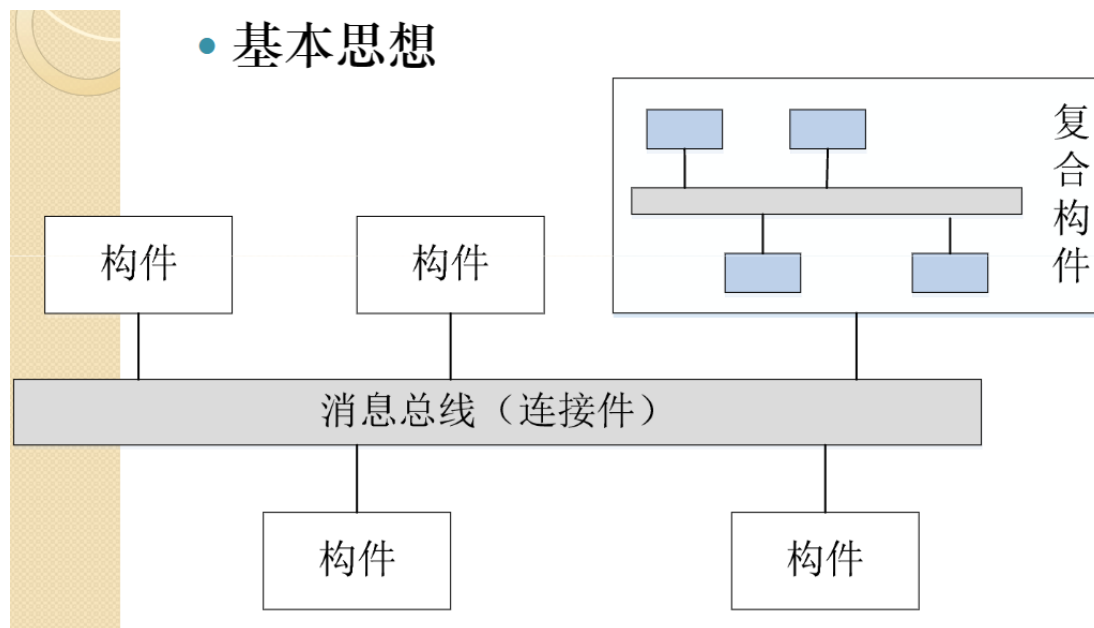
背景：分布式与网络、事件驱动的图像化编程、硬件总线技术

各个组件挂接在消息总线上，向总线登记感兴趣的消息类型。

连接件：消息总线

优点：降低耦合、增强组件重用、易维护

缺点：系统重用性低



MVC风格

MVC结构主要包括模型、视图和控制器三部分。

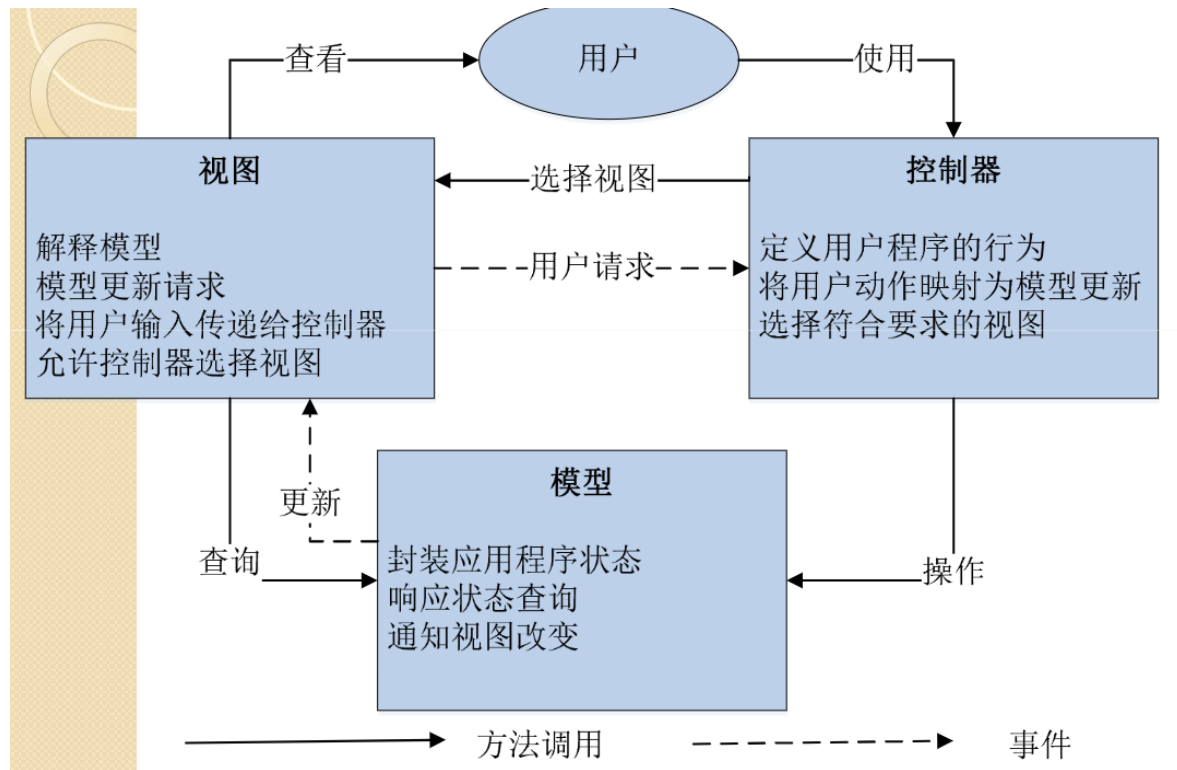
模型：封装了问题的核心数据、逻辑关系和计算功能，提供了处理问题的操作过程和计算功能，提供了处理问题的操作过程

视图：视图是模型的表示，提供了交互界面，为用户显示模型信息

控制器：控制器负责处理用户与系统之间的交互，为用户提供操作接口

优点：多视图与一模型对应，同步性好、具有一定的移植性、易修改

缺点：系统设计复杂、视图与控制器重用性低、访问效率低（需要视图访问多个接口）



软件架构模式比架构风格更加面向问题。

Unit 6 软件架构与敏捷开发

敏捷开发基本理念：

- 个体和互动>过程和工具
- 可运行的软件>详尽的文档
- 客户合作>合同谈判
- 响应变化>遵循计划

软件架构与敏捷开发的关系

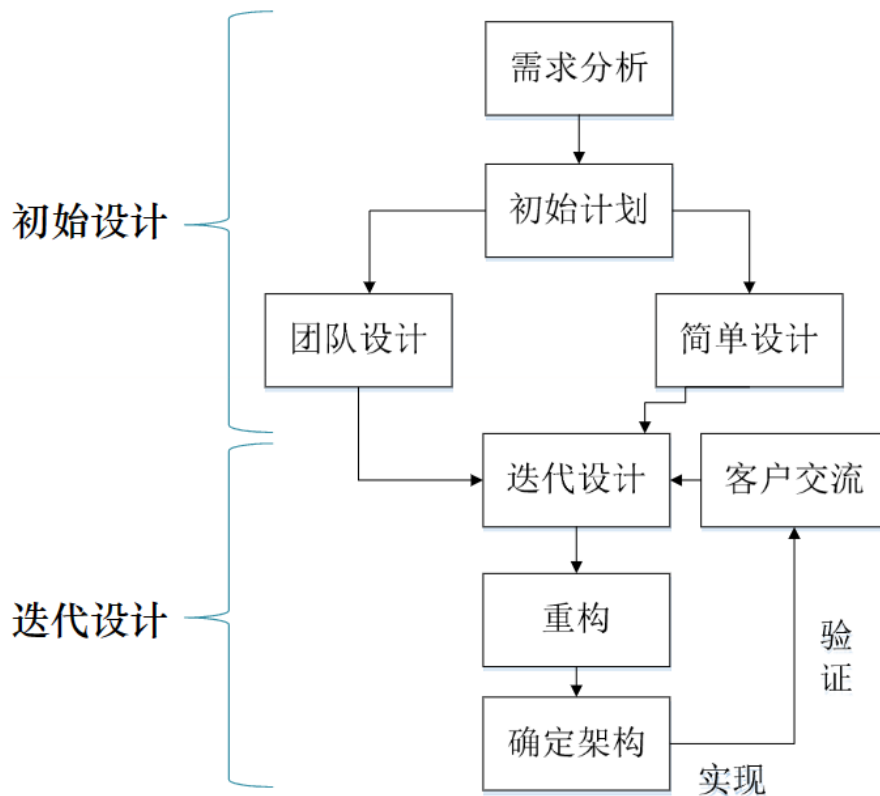
软件架构与敏捷开发的出发点是一致的，都是一个权衡的过程，为了提高软件开发效率。

敏捷开发也需要重视软件架构。

敏捷开发改变了软件架构的设计方式，传统架构变成种子架构设计+详细架构设计。把传统软件开发前期的详细架构设计，分散到了整个敏捷开发软件过程中，以达到提高效率、减少风险的目的。

（先做重要的、充分交流、及时验证、趁早修改、不轻易修改）

6.3 敏捷开发过程中的软件架构设计



敏捷软件架构的一般设计过程

需求分析：分成初始阶段需求分析和迭代阶段需求分析

（规划式设计）初始设计：确定系统的基本流程、结构、划分，得到一个原始架构。

（演进式设计）迭代过程：相对稳定，每次基于上一次迭代的结果。使架构越来越强壮。

敏捷设计思想在软件架构的体现：团队设计和简单设计

Unit 8 软件架构建模方法

简答

成功的软件架构应有的品质：

- 良好的**模块化**
- 适应**功能需求的变化**
- 对**系统**的动态运行有良好的规划
- 对**数据**的良好规划
- 明确、灵活的**部署**规划

软件架构对需求分析的影响

- 否则会有建立需求规约与架构的映射的困难
- 需求与系统设计的一致性
- 更可靠的需求规约

基于体系结构的软件设计方法（ABSD）

目的：组织最早的设计策略，做出功能划分和质量需求，不需要实际的组件

ABSD方法的步骤

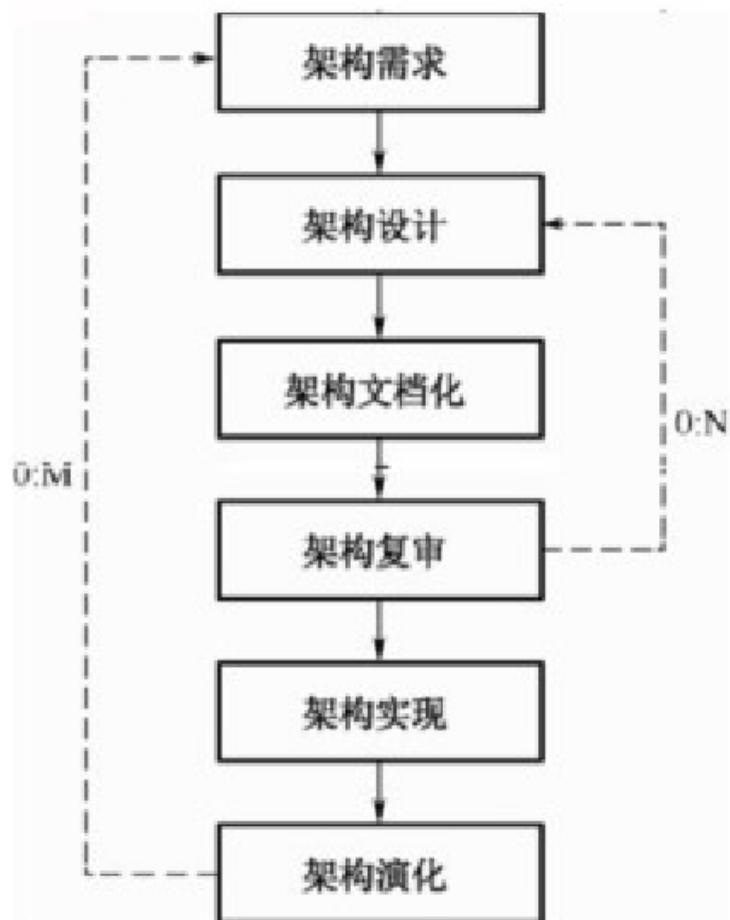


图 基于架构的软件开发模型

架构需求：功能分解

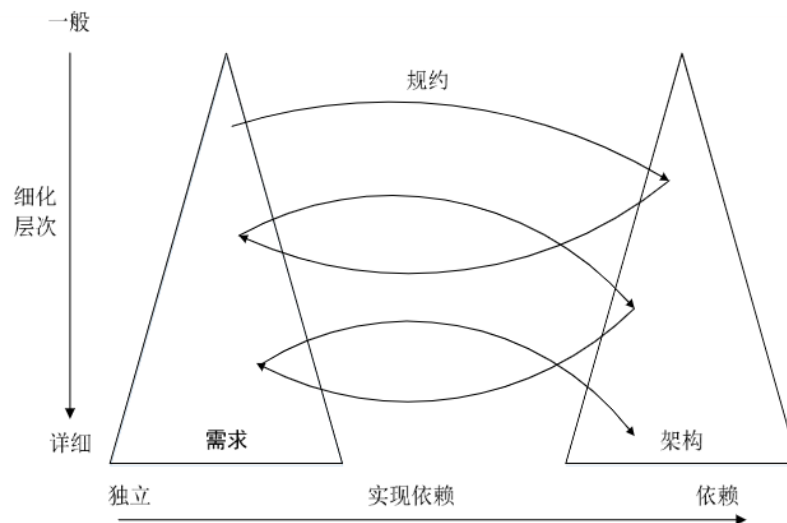
架构设计：选择架构风格、分配功能、细化、校验

架构文档化：**架构需求规格说明**和**测试架构需求的质量设计说明书**

架构与需求协调演化

软件需求和软件架构两者是相辅相成的关系，一方面软件需求影响软件架构设计，另一方面软件架构帮助需求分析的明确和细化。在发展需求和架构规约同时，从解决方案中分离出新的需求和架构规约。

- 需求与架构的互相影响可以看成是一个螺旋的过程，也是一个双峰的过程。



36

从架构设计到详细设计

架构解决全局性问题，详细设计解决局部问题。

软件架构映射到详细设计时经常出现的问题：

- 缺失重要架构视图，片面强调功能需求。
 - 针对遗漏的架构视图进行设计
- 不够深入，架构设计方案过于笼统，基本还停留在概念性架构的层面，没有提供明确的技术蓝图
 - 将设计决策细化到和技术相关的层面。
- 缺失层次之间的交互接口和交互机制，只进行职责划分
 - 明确各层之间的交互接口和交互机制
- 过度设计
 - 切忌过度设计

MDA（基于模型驱动软件架构）

MDA的基本思想：将软件系统分成模型和实现两部分。模型是对系统的描述，与实现技术无关。实现是利用特定技术在特定平台或环境中对模型的解释。

MDA的过程：用计算无关模型CIM 捕获需求、创建平台无关模型PIM、将PIM转化成为一个或多个平台特定模型PSM、将PSM 转化为代码

MDA的好处：将模型与实现分离后，能够很好的适应技术易变性。只需要针对技术、平台编写相应代码即可变换。

软件架构视图：概念视图、模块视图、执行视图（映射到平台）、代码视图（具体代码）

架构设计原则

- 一般设计原则
 - 商业原则
 - 数据原则

简答

- 应用程序原则
- 技术原则
- 关键设计原则
 - 关注点分离——将程序分解为相对无关的、功能重叠部分尽可能少的一个个模块，达到高内聚和低耦合的目的
 - 单一职责——每个组件或模块应负责一个特定特征或功能，或者内聚功能的集合
 - 最少知识原则——一个组件或对象应该对其他组件或对象的内部细节尽可能少的了解
 - 不重复自身原则——只须在一个地方指定意图
 - 尽量减少前期设计——只做必要的设计

简答、问答、选择

架构设计的主要威胁：

- 被忽略的重要非功能需求——全面认识需求
- 频繁变化的需求——关键需求决定架构
- 考虑不全面——多视图探寻架构
- 不及时的架构验证——尽早验证架构
- 较高的创造性的架构比重——合理分配经验架构与创新架构比重
- 架构的低可执行性——验证架构的可执行性

Unit 9 软件架构质量

软件架构质量分为内部质量和外部质量。

内部质量（直接）：开发过程中的软件架构模型、数据、描述文档和视图的质量等

外部质量（间接）：运行过程中的系统的性能、可靠性、安全性等

软件架构质量的意义：

最终软件产品质量、早发现早解决、软件架构自身存在着很高的质量需求

软件架构与软件的质量属性是相辅相成的。

一方面软件架构本身具有质量，高质量的软件架构设计会带来高质量的软件产品

另一方面，软件最终产品的质量可以间接地反映软件架构的质量，可以进一步指导软件架构的演化和优化等

软件质量：

- 性能
- 可用性
- 可靠性
- 安全性
- 易用性
- 可更改性
- 可移植性
- 可重用性
- 可集成性
- 可测试性

软件架构的质量：

- 内部质量（直接）

- 可维护性（纠正错误、提升性能、适应环境）
 - 改正性维护
 - 适应性维护
 - 完善性维护
 - 预防性维护
 - 可重用性（组件重用）
 - 可移植性（不同计算平台）
 - 可集成性（组件协同工作）
 - 可测试性（测试发现缺陷）
 - 外部质量（间接）
 - 性能
 - 可用性（正常运行时间比例）
 - 可靠性（容错、健壮性）
 - 安全性
 - 易用性
-

软件架构评估

软件架构评估的**目的**是判断该架构是否实现了风险承担者的质量需求

步骤：

- 准备评估文档
- 确定风险承担者
- 确定参与人
- 架构明确后、具体实施前开始评估

评估技术

- 基于问卷调查或检查表的软件架构评估技术（早）
 - 优点：灵活
 - 不足：主观
- 基于场景的软件架构评估技术（刺激、场景、响应）（中）
- 基于度量的软件架构评估技术（后）

Unit 10 软件架构测试方法

基于场景的评估方式

思想：分析软件体系结构对场景也就是对系统的使用或修改活动的支持程度，从而判断该体系结构对这一场景所代表的质量需求的满足程度

优点：考虑了所有人的质量要求、展现功能与结构的映射、展现场景支持程度

不足：特定于领域、对实施者要求了解领域以及架构

ATAM评估方法

敏感点：一个或多个构件的**特征**，敏感点影响一个质量属性

权衡点：多个质量属性的敏感点，权衡点影响多个质量属性

场景：采用刺激、环境和响应三方面来对场景进行描述

- 刺激：风险承担者的动作
- 环境：系统的状况
- 响应：风险承担者的结果

ATAM的步骤

1. 陈述

1. ATAM方法的陈述：评估负责人
2. 商业动机的陈述：项目经理或系统客户
3. SA的陈述：系统设计人员

2. 调查与分析

4. 确定体系结构方法
5. 生成质量属性效用树
 - 确定质量属性目标
 - 质量属性细化为场景
 - 设置优先级
6. 分析体系结构方法
 - 质量需求与架构是否匹配

3. 测试

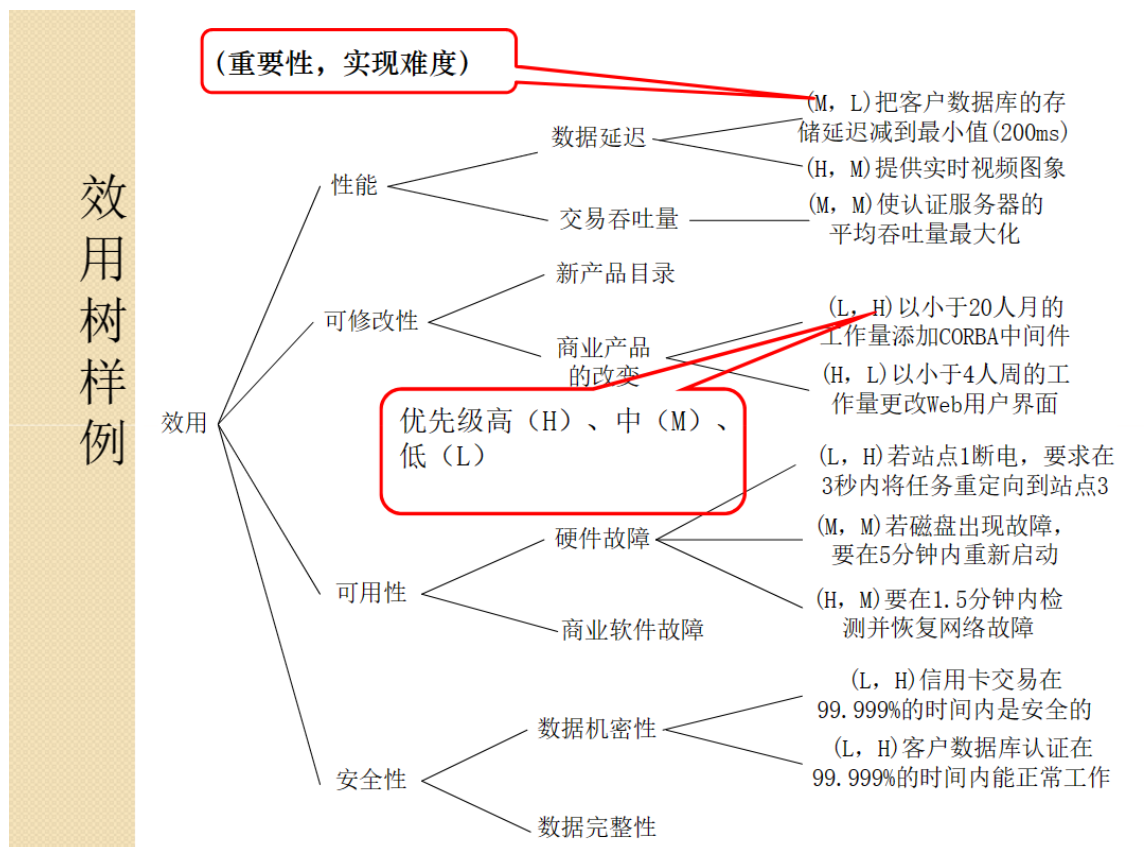
7. 集体讨论并确定场景优先级
8. 分析体系结构方法

4. 形成报告

9. 结果的表述

时间上分成4个阶段：

- 建立阶段——合作准备
- 1~6步——获取体系结构并分析
- 7~9步——验证
- 后续——报告



场景: S12 (检测主 CPU 故障并恢复系统)

属性: 可用性

环境: 正常操作

刺激: CPU 失效

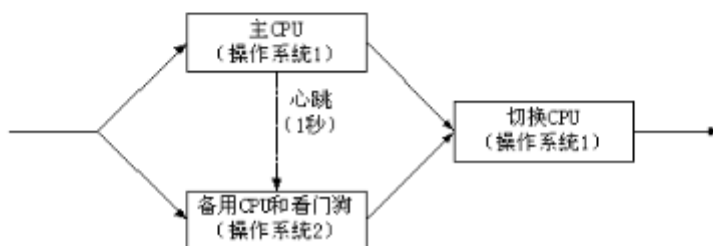
响应: 可用切换概率为 0.999999

体系结构决策	风险	敏感点	权衡点
备用 CPU	R8	S2	
无备用数据通道	R9	S3	T3
看门狗 (watchdog)		S4	
心跳 (heartbeat)		S5	
故障切换路由		S6	

推理:

- (1) 通过使用不同的硬件和操作系统, 保证没有通用模式故障;
- (2) 完成恢复时间最多不超过 4 秒 (也就是一个运算状态时间);
- (3) 基于心跳和看门狗的速度, 保证在 2 秒钟内能检测到故障;
- (4) 看门狗简单的和可靠的 (已被证实过);
- (5) 由于没有备用数据通道, 可用性需求可能存在风险

体系结构图:



SAAM

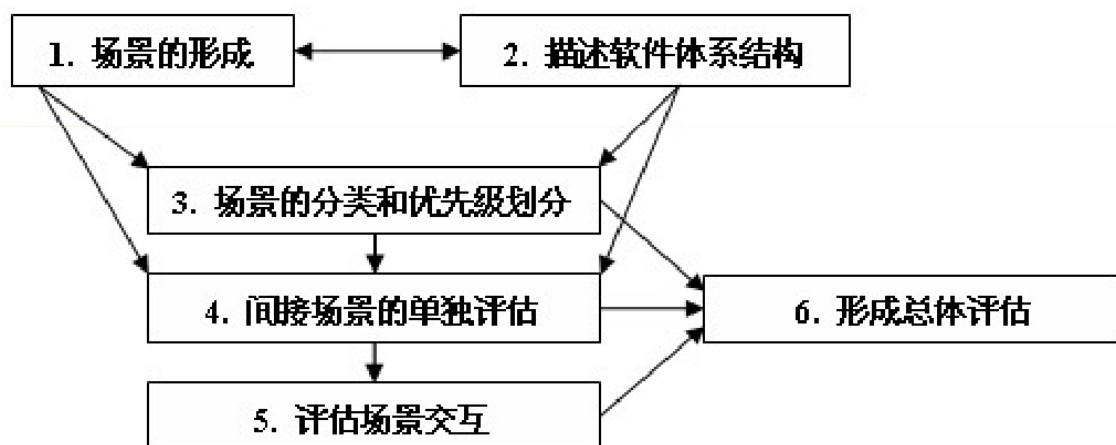
主要分析体系结构的可修改性也可以对系统属性及系统功能进行分析。

评估方法:

1. 场景的形成
2. 体系结构的表述

3. 场景的分类和优先级的确定
4. 对场景的单个评估
5. 场景相互作用的评估
6. 形成总体评估

* SAAM评估步骤



SAAM考查的是SA单独的质量属性，而ATAM提供从多个竞争的质量属性方面来理解SA，使用户能认识到在多个质量属性目标间权衡的必要性

Unit 11 软件架构度量与评估

为什么需要进行软件架构的度量？

从架构出发，可以更有效地把握整个软件系统的设计结构和版本更新问题。

基于架构的度量可以在设计早期发现软件架构的不合理之处

基于架构的度量可以对架构的演化程度进行评估和预测

架构度量是一种静态的度量方法，不需要执行软件或者进行仿真，具有成本低、容易实现且不依赖于程序特定的运行环境的优点

架构演化中可以使用多版本的架构度量技术。首先获取各个版本的分析模型，然后确定不同版本模型中元素之间的对应关系，发现相互之间的基本差异，然后在此基础上得到高层的演化模式或趋势分析结果。

架构度量有助于推动架构演化，验证架构演化是否满足驱动因素。

Unit 12 软件架构演化与维护

软件架构的生命周期：初始设计、实际使用、修改完善（软件架构的演化与维护过程）、退化弃用

软件架构演化与软件架构的定义有关。

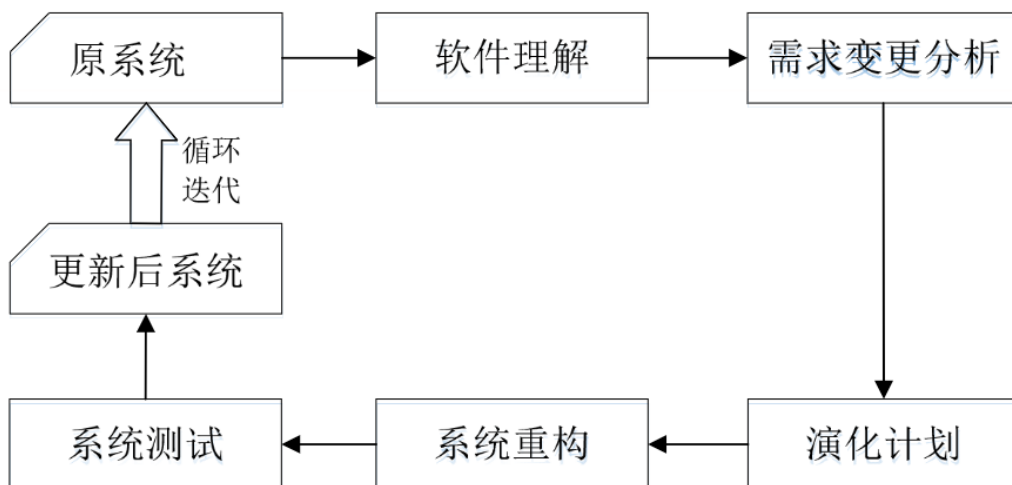
软件架构演化可分为软件架构静态演化和软件架构动态演化。

- 软件架构静态演化
 - 指系统停止运行期间的修改和更新，即一般意义上的修复和升级。发生在软件架构的设计、实现和维护过程中，软件系统还未运行或者处在运行停止状态

- 必要性

- 设计时演化需求：在架构开发和实现过程中**对原有架构进行调整**，保证软件实现与架构的一致性以及软件开发过程的顺利进行
- 运行前演化需求：软件发布之后由于**运行环境的变化**，需要对软件进行修改升级，在此期间软件的架构同样要进行演化

- 一般过程



- 软件理解——分析架构、提取系统抽象表示
- 需求变更分析——需求的变化
- 演化计划——确定范围、成本、计划
- 系统重构——重构
- 系统测试——测试

- 软件架构动态演化

- 发生在软件系统运行过程中

- 必要性

- 软件内部执行所导致的体系结构改变
- 软件系统外部的请求对软件进行的重配置

- 内容

- 属性改名
- 行为变化——软件行为改变（例如加密协议）
- 拓扑结构改变——（修改组件）
- 风格变化——（很有限的）

- 两种主要技术

- 动态软件架构

- 动态重配置

- 四种模式

- 主从模式——“主”组件接收客户端的服务请求，它将工作划分给“从”组件，然后合并、解释、总结或整理从组件的响应
- 中央控制模式——一个中央控制器会控制多个组件，其状态图会维持两个状态，分别标识中央控制器是否处于空闲状态
- 客户端-服务器模式——客户端发起的事务完成之后可以添加或删除客户端组件
- 分布式控制模式——系统的功能整合在多个分布式控制组件之中

- 成本控制原则——演化成本要控制在预期的范围之内，也就是演化成本要明显小于重新开发成本
- 进度可控原则——架构演化要在预期时间内完成，也就是时间成本可控
- 风险可控原则——架构演化过程中的经济风险、时间风险、人力风险、技术风险和环境风险等必须在可控范围内
- 主体维持原则——所有其他因素必须与软件演化协调，开发人员、销售人员、用户必须熟悉软件演化的内容，从而达到令人满意的演化。因此，软件演化的平均增量而达到令人满意的演化。因此，软件演化的平均增量的增长须保持平稳，保证软件系统主体行为稳定
- 系统总体结构优化原则——架构演化要遵循总体结构优化原则，使得演化之后的软件系统整体结构（布局）更加合理
- 平滑演化原则——在软件系统的生命周期里，软件的演化速率趋于稳定，例如相邻版本的更新率相对固定。
- 等等共18个原则

架构维护需要对软件架构的演化过程进行追踪和控制，保障软件架构的演化过程能够满足需求

Unit 13 软件架构重构

什么是软件架构重构？为什么要进行软件架构重构？

软件架构重构是在不改变软件的功能和外部可见性的情况下，为了改善软件的结构，提高软件的可读性、为了改善软件的结构，提高软件的可读性、可扩展性、可重用性、可维护性等质量属性的情况下而对其进行的改造。

主要过程是重构点识别与对重构点实施重构操作。

软件重构是进行软件优化和提高软件质量的重要手段之一。非功能需求改变。软件架构级 坏味道、设计技术债

基于度量的软件架构重构

该重构发生在一次演进过程结束之后，将**软件质量**引入 重构过程，通过**架构度量和评估**结果中发现的**架构缺陷**确定 **重构对象和重构范围**，并提出**重构建议**。

如何通过软件架构度量原则获取软件重构需求

选取相关的[软件架构持续演进原则]，形成达成性度量方法，通过度量评估该软件架构持续 演进原则的达成情况。通过这些度量就可以计算得出目前相邻版本之间的演化原则的达成度。取值范围都是 $[-1, 1]$ ，越大越好。

识别度量结果中未达成的架构持续演进原则子指标，并分析对应公式查看能否通过重构完善。

如何通过软件架构度量原则进行软件重构建议。

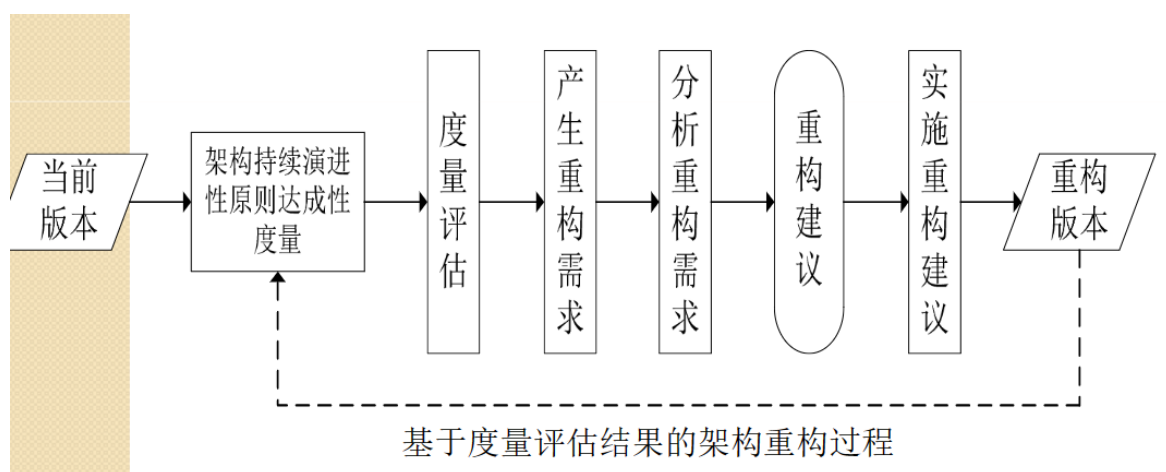
针对不同重构需求下的不同架构缺陷选择恰当的方法对系统进行重构，重构方法应明确重构 对象以及重构操作。

涉及到组件层次的相关指标需要对组件及组件之间的依赖关系进行修改。

涉及到代码层次的相关指标需要对代码进行修改。

架构持续演进原则子指标	重构建议
组件规模最小化原则	拆分组件或提取组件
模块独立演进原则	降低文件之间耦合
复杂性可控原则	降低文件圈复杂度
主体维持原则	定位组件具体变化反馈用户
平滑演进原则	定位规模具体变化反馈用户
外部接口稳定原则	定位接口具体变化反馈用户

重构过程



- 度量评估——选取相关的[软件架构持续演进原则]，形成达成性度量方法，通过度量评估该软件架构持续演进原则的达成情况。通过这些度量就可以计算得出目前相邻版本之间的演化原则的达成度。取值范围都是[-1,1]，越大越好。
- 产生重构需求——识别度量结果中未达成的架构持续演进原则子指标，并分析对应公式查看能否通过重构完善。
- 分析重构需求：

度量原则	重构背景	重构主要原因
主体维持原则	(1) 组件依赖图中的组件没有发生变动，组件之间的依赖关系发生了较大变化； (2) 组件依赖图中的组件发生了变化，从而导致组件之间的依赖关系也发生了变化。	组件和组件之间的依赖关系变动太大
平滑演进原则	(1) 组件本身规模发生了较大的变动； (2) 组件增加或删除。	组件规模平均变化率过大
组件规模最小化原则	(1) 软件规模不变，组件数降低； (2) 组件数不变，软件规模增大； (3) 软件规模减少，组件数减少，且软件规模的减少比例小于组件数减少的比例； (4) 软件规模增大，组件数增大，且软件规模的增加比例大于组件数增大的比例	组件平均规模过大

- 重构建议——针对不同重构需求下的不同架构缺陷选择恰当的方法对系统进行重构，重构方法应明确重构对象以及重构操作。
涉及到组件层次的相关指标需要对组件及组件之间的依赖关系进行修改。
涉及到代码层次的相关指标需要对代码进行修改。

Unit 14 软件架构腐蚀和对策

架构腐蚀的含义——指预期软件架构或概念软件架构与实际软件架构之间的偏离。这种偏离更多的是源自日常的软件修改

三种预防控制方法：

- 腐蚀最小化
 - 面向过程的架构一致性策略
 - 统一而严格地编写软件架构设计文档——参考依据
 - 通过软件架构分析进行架构评估和复查——全面揭示
 - 软件架构依从性监控——验证
 - 软件架构依赖性分析揭示特定系统中违背软件架构规则的情况——查找
 - 架构演化管理策略——研究架构和软件的共同演化
 - 架构设计实施策略——采用形式化或半形式化的方法对目标系统进行建模（减小误差）
- 腐蚀预防
 - 联动架构策略——建立架构及其实现的连续关联，既独立地演变，又遵守一致性
 - 自适应策略——预见潜在的变化
- 腐蚀修补
 - 架构恢复策略——从源代码或其他架构文档中抽取架构，为腐蚀的检测、评价和修复建立基础
 - 架构发现策略——发现新的架构以备重用
 - 研究系统用例，从系统需求来构建预期的架构
 - 从源码生成类图，对类图进行聚类抽取另一个架构
 - 两个架构整合成新架构
 - 架构协调策略——修补实际架构以使其符合恢复或发现的预期架构

Unit 15 软件架构获取与恢复

软件架构恢复——从工程项目中获取所需的架构信息，恢复出架构的组成元素。

软件架构恢复的意义——从源代码或其他架构文档中抽取架构，为腐蚀的检测、评价和修复建立基础

软件架构恢复的三种形式——自顶向下、自底向上、混合过程

软件架构恢复的过程

- 信息提取
 - 静态分析
 - 动态分析
- 信息表达

Unit 16 软件架构技术债

技术债的含义——开发人员为了加速软件开发（或 修改），或是由于自身经验的缺乏，有意或 无意地在应该采用最佳方案时进行了妥协， 使用了短期内能加速软件开发的方案，从而 在未来给自己带来的额外开发负担

技术债的分类

- 代码债
- 设计债
 - 识别方法
 - ASA问题识别（工具分析源码）
 - 代码坏味识别（面向对象系统）
 - 设计模式污垢识别（与模式无关的代码）
 - 模块化冲突（不同模块但一起变化的组件存在差异）
- 测试债
- 文档债

技术债的成因

- 进度压力
- 软件设计师缺乏足够的经验和技巧
- 不注重设计原则的应用
- 缺乏对设计坏味和重构的意识
- 开发中有意采用非最优的选择

技术债的处理

- 发现技术债
- 管理技术债（识别、评估）
- 偿还技术债

Unit 17 软件架构坏味道

坏味道分为代码坏味道和架构坏味道

代码坏味道——代码是不稳定或者有一些潜在问题的，那么该段代码往往会包含一些明显的不太好的痕迹

- 可能成因
 - 代码层面
 - 类层面
 - 方法层面

架构坏味道——对系统生命周期特性产生消极影响的架构设计

- 可能成因
 - 不合 适的解决方案
 - 错误的粒度层次下应用了某个设计抽象
 - (1) 连接件嫉妒 (Connector Envy) ;
 - (2) 过度分散的功能 (Scattered Functionality) ;
 - (3) 模糊接口 (Ambiguous Interface) ;
 - (4) 无关的相邻连接件 (Extraneous Adjacent Connector) ; Connector) ;
 - (5) 砖关注过载 (Brick Concern Overload) ;
 - (6) 砖使用过载 (Brick Use Overload) ;
 - (7) 砖循环依赖 (Brick Dependency Cycle) ;
 - (8) 未使用接口 (Unused Interface) ;
 - (9) 重复的组件功能 (Duplicate Component Functionality) ;
 - (10) 组件嫉妒 (Component Envy) ;
 - (11) 连接件链 (Connector Chain)

Unit 18 软件脆弱性

软件脆弱性的定义——软件脆弱性是导致 破坏系统安全策略的系统安全规范、系统 设计、实现和内部控制等方面的弱点

软件脆弱性的特点：

- 脆弱性是软件系统中隐藏的一个弱点，**本身不会引起危害**，但被利用后会产生严重的安全后果
- 在软件开发过程中，**自觉或不自觉引入**的逻辑错误是大多数脆弱性的根本来源
- **与具体的系统环境密切相关**，系统环境 的任何差异都有可能 导致不同的脆弱性问题
- 旧的脆弱性得到修补或纠正的同时可能 会引入**新的脆弱性**

软件脆弱性的成因——软件开发过程或系统配置过程的错误

软件脆弱性的生命周期：

- 脆弱性的引入阶段
- 产生破坏效果阶段
- 修补阶段

软件**架构**脆弱性——架构设计存在一些明显的或者隐含的缺陷，被攻击后会导致质量下降。

软件**架构**脆弱性成因——软件架构的风格 (style) 和模式 (pattern) 有关 (分层架构等)